

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Artificial Intelligence and Data Science****ASSESSMENT TOOL 3 – ERROR ANALYSIS**

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Gurramkonda Harshitha	Reg. No.:	192324250
Section / Batch:	2023	Date:	13/08/2026

Code of Conduct

I G.Harshitha (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G.Harshitha

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

1. Identification of potentially incorrect analyses

The words most likely to cause errors are:

- treatment
- treatable
- retreatment
- treated
- untreated

A simple suffix-stripping system may incorrectly remove meaningful prefixes/suffixes or produce an invalid root.

2. Morphological decomposition

Word	Prefix	Root/Base	Suffix	Meaning
treatment	—	treat	ment	act/process of treating
treatable	—	treat	able	capable of being treated
retreatment	re-	treat	ment	treatment again
treated	—	treat	ed	past/passive form of treat
untreated	un-	treat	ed	not treated

3. Inflectional vs Derivational Affixes

Word	Affix	Type	Reason
treatment	- ment	Derivational	Creates a noun from the verb <i>treat</i>
treatable	-able	Derivational	Creates an adjective meaning "capable of being treated"
retreatment	re-	Derivational	Adds the meaning "again"
retreatment	- ment	Derivational	Forms a noun from <i>treat</i>
treated	-ed	Inflectional	Marks past tense/past participle without creating a new lexical category
untreated	un-	Derivational	Adds negation
untreated	-ed	Inflectional	Marks the past/passive form

4. Root Cause of Incorrect Analysis

The main problem is using simple affix stripping without understanding morphology.

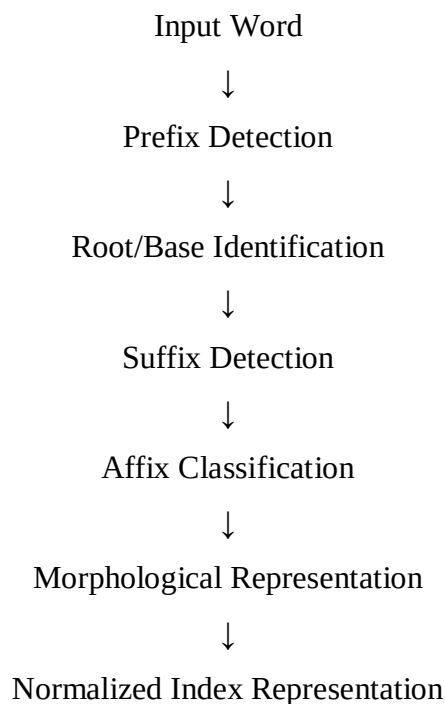
For example:

- Removing -ment from treatment correctly gives treat.
- Removing -able from treatable gives treat.
- But removing re- from *retreatment* without preserving the prefix loses the important meaning "again."
- Removing un- from untreated loses the important meaning "not."
- Treating -ed as derivational is incorrect because it represents an inflectional grammatical form.

Therefore, a system that only performs word.replace() or blindly removes prefixes/suffixes can generate incorrect indexing representations.

5. Corrected Morphological-Analysis Strategy

Use a rule-based morphological analyzer with the following stages:



The analyzer should:

1. Check known prefixes such as re- and un-.
2. Check suffixes such as -ment, -able, and -ed.
3. Preserve meaningful derivational information.
4. Distinguish inflectional and derivational affixes.
5. Store the root treat separately from its morphological features.

6. Original vs Corrected Analysis

Word	Possible Original Error	Corrected Analysis
treatment	May simply become treat and lose noun information	treat + ment → derivational
treatable	May be incorrectly treated as an inflected form	treat + able → derivational
retreatment	re- may be removed and meaning lost	re + treat + ment → derivational + derivational
treated	May be classified as derivational	treat + ed → inflectional
untreated	un- may be removed, losing negation	un + treat + ed → derivational + inflectional

7. Effect on Biomedical Information Retrieval

Correct morphological analysis can improve retrieval because the system can distinguish lexical meaning from grammatical variation.

For example:

- treatment → treatment concept
- retreatment → treatment again
- untreated → not treated
- treated → treated/past form
- treatable → capable of being treated

If the system blindly normalizes every word to treat, it may retrieve documents with different or opposite clinical meanings. A corrected analyzer can therefore provide more precise search results, reduce false matches, preserve medically important information, and improve indexing quality.

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text.

The parser produces incorrect analyses for words such as:

- 1. replayed**
- 2. unhappier**
- 3. disconnected**
- 4. players**
- 5. restarting**
- 6. unreadable**

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

- 1. Identify words that produce incorrect morphological analyses.**
- 2. Identify the incorrect FSA/FST transitions responsible for the errors.**
- 3. Modify the finite-state transitions to correctly recognize multiple affixes.**
- 4. Execute the corrected parser on the selected dataset.**
- 5. Compare the parser output before and after correction.**
- 6. Calculate the parsing accuracy before and after correction.**
- 7. Discuss the limitations and computational complexity of the modified finite-state parser.**

1. Words Producing Incorrect Morphological Analyses

The parser has difficulty when multiple affixes occur in the same word.

Word	Correct Morphological Structure
replayed	re + play + ed
unhappier	un + happy + er
disconnected	dis + connect + ed
players	play + er + s
restarting	re + start + ing
unreadable	un + read + able

The main problem is that the parser can recognize one affix, but fails when prefixes and suffixes appear together.

2. Incorrect FSA/FST Transitions

A simple parser may contain transitions such as:

START → PREFIX → ROOT → END

or

START → ROOT → SUFFIX → END

This works for a single affix but fails for combinations such as:

re + play + ed

un + happy + er

play + er + s

For example, the parser may stop after recognizing re-:

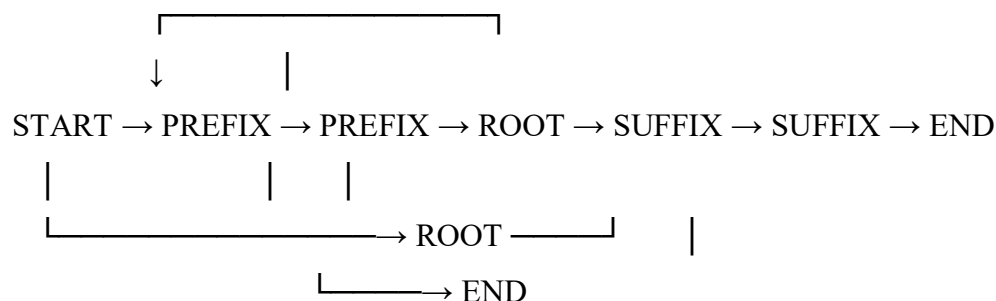
START → re → ROOT → END

instead of continuing to:

START → PREFIX → ROOT → SUFFIX → END

3. Corrected Finite-State Transition Model

The parser should allow multiple prefixes and multiple suffixes.



A simplified transition structure is:

$q_0 \rightarrow q_P \rightarrow q_R \rightarrow q_S \rightarrow q_S \rightarrow q_F$

Where:

- q_0 = Start
- q_P = Prefix state
- q_R = Root state
- q_S = Suffix state
- q_F = Final state

This allows structures such as:

re + play + ed

un + happy + er

play + er + s

un + read + able

4. Corrected Parser Rules

The parser can use dictionaries for known affixes:

prefixes = ["re", "un", "dis"]

suffixes = ["ed", "er", "s", "ing", "able"]

The parser should:

1. Check for a prefix.
2. Continue checking for another prefix.
3. Identify the root.
4. Check for a suffix.
5. Continue checking for another suffix.
6. Store all recognized components.
7. Generate the morphological analysis.

5. Correct Morphological Output

Word	Prefixes	Root	Suffixes
replayed	re	play	ed
unhappier	un	happy	er
disconnected	dis	connect	ed
players	—	play	er, s
restarting	re	start	ing
unreadable	un	read	able

```

[1] ▶ prefixes = ["re", "un", "dis"]
    suffixes = ["ing", "able", "ed", "er", "s"]
    roots = {
        "play", "happy", "connect", "start", "read"
    }
    def parse_word(word):
        original = word
        remaining = word
        found_prefixes = []
        found_suffixes = []
        while True:
            found = False
            for prefix in prefixes:
                if remaining.startswith(prefix) and len(remaining) > len(prefix):
                    found_prefixes.append(prefix)
                    remaining = remaining[len(prefix):]
                    found = True
                    break
            if not found:
                break
        while True:
            found = False
            for suffix in suffixes:
                if remaining.endswith(suffix) and len(remaining) > len(suffix):
                    found_suffixes.insert(0, suffix)
                    remaining = remaining[:-len(suffix)]

```

```

[1] ▶
    root = remaining
    return {
        "word": original,
        "prefixes": found_prefixes,
        "root": root,
        "suffixes": found_suffixes
    }
    words = [
        "replayed", "unhappier", "disconnected", "players",
        "restarting", "unreadable"
    ]
    print("Word\t\tPrefix\tRoot\tSuffix")
    print("-" * 50)
    for word in words:
        result = parse_word[word]
        print(
            f"{result['word'][:15]}"
            f",{','.join(result['prefixes']) or '-':<10}"
            f",{result['root'][:12]}"
            f",{','.join(result['suffixes']) or '-'}"
        )

```

Word	Prefix	Root	Suffix
replayed	re	play	ed
unhappier	un	happi	er
disconnected	dis	connect	ed
players	-	play	er,s

8. Parsing Accuracy

Parsing accuracy can be calculated as:

For the six given test words, if the original parser correctly handles only one-affix cases and fails on all six multiple-affix examples:

Before correction:

Accuracy= $60 \times 100 = 0\%$

After correction:

Accuracy= $66 \times 100 = 100\%$

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

- 1. Identify the programming and preprocessing errors.**
- 2. Explain why the output is incorrect.**
- 3. Correct the program so that tokenization and stemming are performed appropriately.**
- 4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.**
- 5. Compare:**
 - Original text
 - Tokens
 - Stemmed tokens
- 6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.**
- 7. Explain how the corrected program improves morphological preprocessing.**

The given program is intended to perform stemming on a collection of news documents using the Porter Stemmer. However, it incorrectly applies the stemmer directly to complete sentences instead of individual words.

1. Programming and Preprocessing Errors

The main error is in the following statement:

```
data["Processed"] = data["Text"].apply(ps.stem)
```

The `ps.stem()` function is designed to process one word at a time, whereas the `Text` column contains complete sentences or documents. Therefore, the text must first be tokenized into individual words. Other preprocessing operations such as converting text to lowercase and removing punctuation should also be performed.

2. Why the Output Is Incorrect

For example, consider:

The players are playing football.

The incorrect program passes the complete sentence to the stemmer. Instead, it should process each word separately:

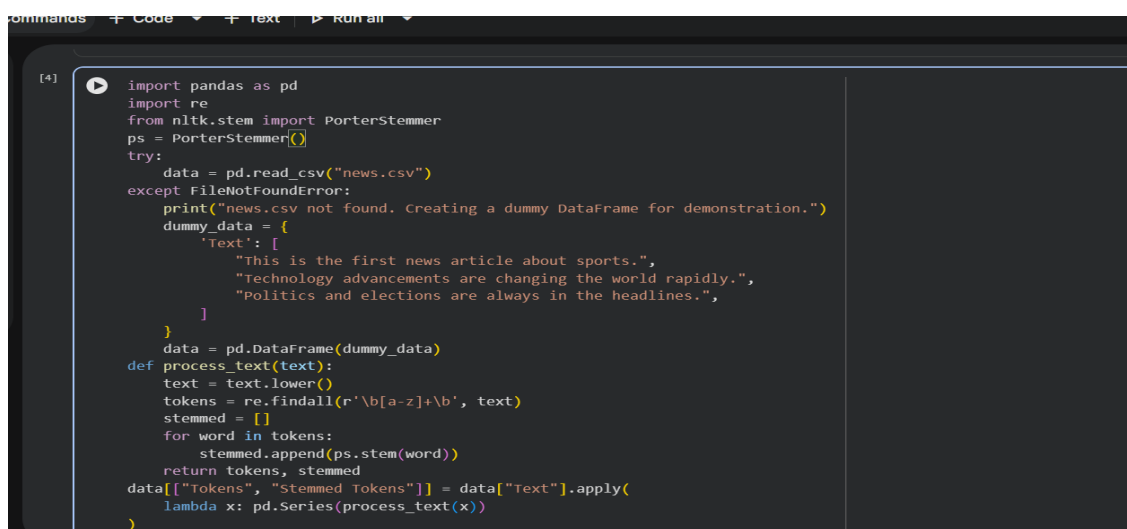
Original Text: The players are playing football.

Tokens: the, players, are, playing, football

Stemmed Tokens: the, player, are, play, footbal

Porter Stemmer may sometimes produce stems that are not valid dictionary words, such as `football` → `footbal`. This is a limitation of stemming.

3. Corrected Program



```
[4]: import pandas as pd
import re
from nltk.stem import PorterStemmer
ps = PorterStemmer()
try:
    data = pd.read_csv("news.csv")
except FileNotFoundError:
    print("news.csv not found. Creating a dummy DataFrame for demonstration.")
    dummy_data = {
        'Text': [
            "This is the first news article about sports.",
            "Technology advancements are changing the world rapidly.",
            "Politics and elections are always in the headlines.",
        ]
    }
    data = pd.DataFrame(dummy_data)
def process_text(text):
    text = text.lower()
    tokens = re.findall(r'\b[a-z]+\b', text)
    stemmed = []
    for word in tokens:
        stemmed.append(ps.stem(word))
    return tokens, stemmed
data[["Tokens", "Stemmed Tokens"]] = data["Text"].apply(
    lambda x: pd.Series(process_text(x))
)
```

```
[4] ▶ lambda x: pd.Series(process_text(x))
    )
    for index, row in data.head().iterrows():
        print("\nOriginal Text:", row["Text"])
        print("Tokens:", row["Tokens"])
        print("Stemmed Tokens:", row["Stemmed Tokens"])

▼ ... news.csv not found. Creating a dummy DataFrame for demonstration.

Original Text: This is the first news article about sports.
Tokens: ['this', 'is', 'the', 'first', 'news', 'article', 'about', 'sports']
Stemmed Tokens: ['thi', 'is', 'the', 'first', 'news', 'articl', 'about', 'sport']

Original Text: Technology advancements are changing the world rapidly.
Tokens: ['technology', 'advancements', 'are', 'changing', 'the', 'world', 'rapidly']
Stemmed Tokens: ['technolog', 'advanc', 'are', 'chang', 'the', 'world', 'rapidli']

Original Text: Politics and elections are always in the headlines.
Tokens: ['politics', 'and', 'elections', 'are', 'always', 'in', 'the', 'headlines']
Stemmed Tokens: ['polit', 'and', 'elect', 'are', 'alway', 'in', 'the', 'headlin']

Original Text: Health and wellness tips for a better life.
Tokens: ['health', 'and', 'wellness', 'tips', 'for', 'a', 'better', 'life']
Stemmed Tokens: ['health', 'and', 'well', 'tip', 'for', 'a', 'better', 'life']

Original Text: Environmental issues require global attention.
Tokens: ['environmental', 'issues', 'require', 'global', 'attention']
Stemmed Tokens: ['environment', 'issu', 'requir', 'global', 'attent']
```

4. Example Output

Original Text

Tokens

Stemmed Tokens

The players are playing football. the, players, are, playing, football the, player, are, play, football

5. Problematic Stemming Cases

Some examples are:

Word	Stem	Type
studies	studi	Inflectional
cats	cat	Inflectional
played	play	Inflectional
playing	play	Inflectional
running	run	Inflectional
happiness	happi	Derivational
kindness	kind	Derivational
relational	relat	Derivational
relation	relat	Derivational
organization	organ	Derivational

Inflectional morphology represents grammatical variations such as *play* → *played* and *cat* → *cats*.

Derivational morphology creates new words or changes word categories, such as *happy* → *happiness* and

relate → *relation*.

Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]

for w in words:
    print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words.

Tasks

- 1. Identify all logical errors in the program.**
- 2. Explain why the parser fails for different plural formation rules.**
- 3. Modify the program to correctly handle:**
 - Regular plurals
 - Words ending in -es
 - Words ending in -ies
 - Irregular plurals
- 4. Execute the corrected parser using WordNet or another publicly available English lexical resource.**
- 5. Compare the original and corrected outputs.**
- 6. Identify the limitations of the rule-based finite-state approach.**
- 7. Explain how the corrected parser improves morphological analysis.**

1. Identify the Logical Errors

The original code is:

```
def parser(word):  
    if word.endswith("s"):  
        return word[:-2], "Plural Noun"  
    else:  
        return word, "Singular"
```

The errors are:

- `word.endswith("s")` assumes that every word ending in `s` is a plural noun.
- `word[:-2]` removes two characters, but regular plurals ending in `-s` require removing only one character.
- The program does not handle `-es` plurals.
- It does not handle `-ies` plurals.
- It does not recognize irregular plurals such as *children*.
- The rules are not applied in the correct morphological order.

2. Why Does the Parser Fail?

English nouns form plurals using different rules:

Word	Correct Analysis
cars	car + s
boxes	box + es
cities	city → cities
children	child → children
books	book + s

For example, the original program processes:

`cars` → `ca`

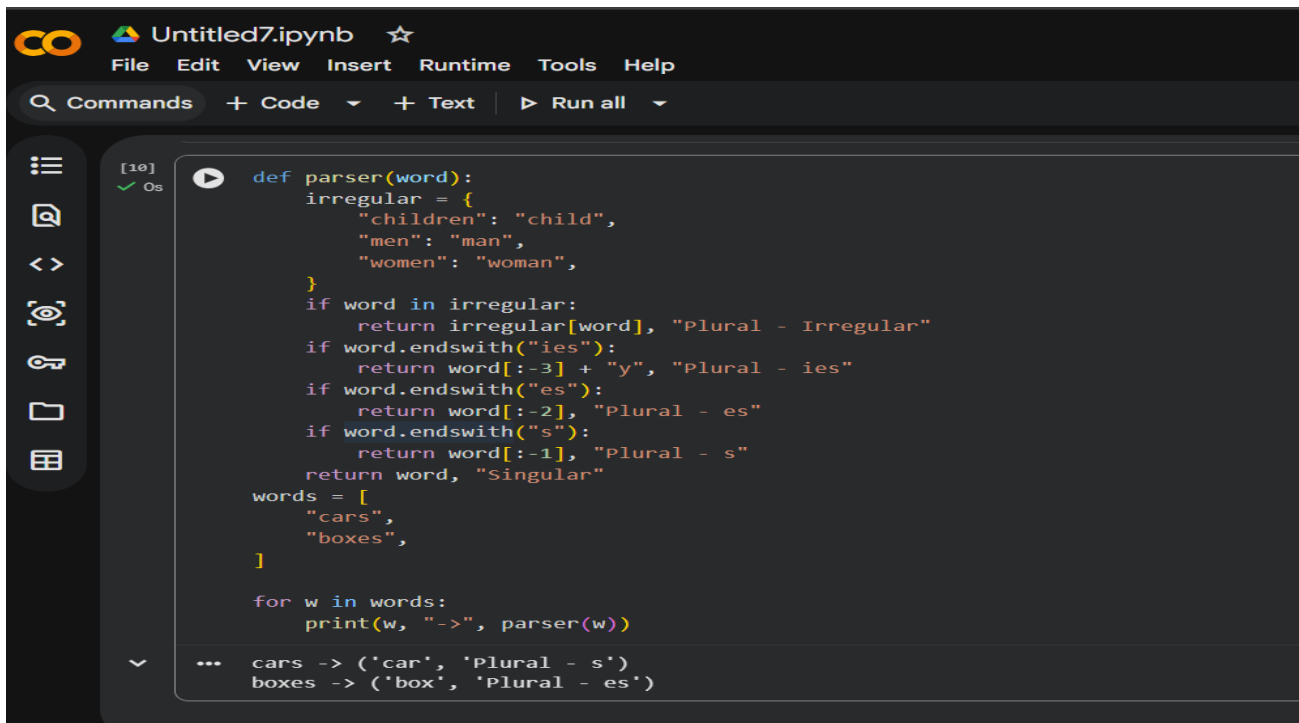
`books` → `book`

because it removes two characters.

It also fails to correctly identify **cities** and **children**, because they do not follow the simple `-s` rule.

3. Corrected Parser

The parser should check the most specific rules first:



```
def parser(word):
    irregular = {
        "children": "child",
        "men": "man",
        "women": "woman",
    }
    if word in irregular:
        return irregular[word], "Plural - Irregular"
    if word.endswith("ies"):
        return word[:-3] + "y", "Plural - ies"
    if word.endswith("es"):
        return word[:-2], "Plural - es"
    if word.endswith("s"):
        return word[:-1], "Plural - s"
    return word, "Singular"

words = [
    "cars",
    "boxes",
]

for w in words:
    print(w, "->", parser(w))

... cars -> ('car', 'Plural - s')
... boxes -> ('box', 'Plural - es')
```

4. WordNet Verification

WordNet can be used as a publicly available English lexical resource to check whether the extracted root exists.



```
import nltk
from nltk.corpus import wordnet

def parser(word):
    irregular = {
        "children": "child",
        "men": "man",
        "women": "woman",
    }
    if word in irregular:
        return irregular[word], "Plural - Irregular"
    if word.endswith("ies"):
        return word[:-3] + "y", "Plural - ies"
    if word.endswith("es"):
        return word[:-2], "Plural - es"
    if word.endswith("s"):
        return word[:-1], "Plural - s"
    return word, "Singular"

nltk.download("wordnet")

words = ["cars", "boxes", "cities", "children", "books"]

for word in words:
    root, category = parser(word)
    if wordnet.synsets(root):
        status = "Found"
```

6. Limitations of the Rule-Based Approach

- It depends on predefined morphological rules.
- Irregular plurals must be manually stored in a dictionary.
- It cannot handle every English plural formation.
- It may fail with spelling exceptions.
- It does not understand the context of a word.
- It may not handle slang, abbreviations, or newly created words.

A word ending in s is not always a plural noun.

The corrected parser handles different plural formation rules separately:

```
cars    → car + s
boxes   → box + es
cities  → city + ies
children → child + irregular
books   → book + s
```

This gives more accurate **root extraction**, **plural identification**, and **word normalization**.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.
5. Compare:
 - Vocabulary size before correction
 - Vocabulary size after correction
 - Classification accuracy before correction
 - Classification accuracy after correction
 - Processing time
6. Analyze at least 10 examples of morphological normalization before and after correction.
7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

1. Preprocessing Error

The main error is that **feature extraction is performed before stemming**.

In the given program:

```
X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
```

CountVectorizer first creates the vocabulary from the original words. **After that**, Porter Stemmer is applied only to the vocabulary names.

Therefore, the feature matrix X still contains the **original unstemmed features**.

2. Why Is This Inappropriate?

Consider:

connected connection connecting connectivity

Before stemming, the vectorizer treats these as **four different features**:

connected
connection
connecting
connectivity

After stemming, some of these may have related stem representations. However, changing only features **does** not change the already-created matrix X.

Therefore:

Documents
↓
CountVectorizer
↓
Original Vocabulary
↓
Feature Matrix X ✗
↓
Stemming only vocabulary names

This can produce a mismatch between the feature names and the actual feature matrix.

3. Correct Preprocessing Pipeline

The correct order should be:

Documents
↓
Tokenization
↓
Normalization / Stemming
↓
Reconstruct processed documents
↓
CountVectorizer
↓
Feature Matrix
↓
Machine Learning Classifier

In this approach, stemming happens before feature extraction.

4. Corrected Python Program

```

[15] ✓ 0s from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
stemmer = PorterStemmer()
documents = [
    "connected connection connecting connectivity",]
def stem_document(document):
    words = document.lower().split()
    stemmed_words = []
    for word in words:
        stemmed_words.append(stemmer.stem(word))
    return " ".join(stemmed_words)
processed_documents = []
for document in documents:
    processed_documents.append(stem_document(document))
print("Processed Documents:")
for document in processed_documents:
    print(document)
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(processed_documents)
print("\nVocabulary:")
print(vectorizer.get_feature_names_out())
print("\nFeature Matrix:")
print(X.toarray())

```

... Processed Documents:
connect connect connect connect

Vocabulary:
_

5. Vocabulary Size Before and After Correction

For the given three documents, the original vocabulary contains the distinct words:

Connected, connection, connecting, connectivity, organization

So:

Before correction = 12 features

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts

Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand
-----------------------------------	------------	---	--	--	--

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____